Denarii Dolor — Unit Test Results

Test plan	Unit Test Plan: Transaction Validation (TC-01 – TC-20)
Build	`main` + the plan's new test cases (uncommitted working tree)
Executed	2026-09-21 11:33 MST (18:33 UTC), host `jabbott-iii-MSI`
Command	`./gradlew testDebugUnitTest createDebugUnitTestCoverageReport`
Tooling	Gradle 8.11.1 · Android Gradle Plugin 8.7.3 · JUnit 4.13.2 · JaCoCo 0.8.12
Verdict	**PASS**: all exit criteria met

---

1. Summary

Measure	Result	Exit criterion	Met?
Plan test cases (TC-01 – TC-20)	**20 / 20 passed**	All pass	✅
Full unit suite	**116 tests, 0 failures, 0 ignored, 6.86 s total**	No regressions	✅
`ValidateTransactionUseCase` line coverage	**97.4 %** (37 / 38 lines)	≥ 90 %	✅
`ValidateTransactionUseCase` branch coverage	**94.4 %** (34 / 36 branches)	Every `failure(...)` reached	✅
`Validators` / `Money` line coverage	**100 %** / **90 %**	Informational	—

Figure 1. Gradle's test report summary for the full unit suite: 116 tests, 0 failures, 100 % successful. This is a screenshot of `app/build/reports/tests/testDebugUnitTest/index.html`.

Test Summary

116	0	0	6.851s
tests	failures	ignored	duration

100%
successful

Packages Classes

Package	Tests	Failures	Ignored	Duration	Success rate
com.denariidolor	60	0	0	0.434s	100%
com.denariidolor.data.local.db	5	0	0	0.033s	100%
com.denariidolor.data.local.preferences	17	0	0	6.109s	100%
com.denariidolor.domain.report	4	0	0	0.023s	100%
com.denariidolor.presentation.ui	21	0	0	0.062s	100%
com.denariidolor.presentation.viewmodel	5	0	0	0.189s	100%
com.denariidolor.util	4	0	0	0.001s	100%

Generated by Gradle 8.11.1 at Sep 21, 2026, 11:33:42 AM

2. Results against the test plan

ID	Test method	Expected	Actual	Time
TC-01	`rejectsZeroOrNegativeAmount`	"Invalid amount" for 0 and −1	✅ Pass	3 ms
TC-02	`rejectsBlankDescription`	"Description cannot be blank"	✅ Pass	1 ms
TC-03	`rejectsNonPositiveDate`	"Invalid date"	✅ Pass	1 ms
TC-04	`rejectsNonPositiveAccountOrCategoryIds`	"Invalid account" / "Invalid category"	✅ Pass	1 ms
TC-05	`failsWhenCategoryOrAccountMissing`	"Category not found" / "Account not found"	✅ Pass	2 ms
TC-06	`transferRequiresDestination`	"Transfer destination is required"	✅ Pass	2 ms
TC-07	`transferFailsWhenDestinationMatchesSource`	failure	✅ Pass	2 ms
TC-08	`transferFailsWhenDestinationAccountMissing`	"Transfer destination account not found"	✅ Pass	1 ms
TC-09	`validTransferPasses`	success	✅ Pass	1 ms
TC-10	`expenseFailsWhenBudgetExceeded`	"Budget threshold violated"	✅ Pass	1 ms

ID	Test method	Expected	Actual	Time
TC-11	`expenseExactlyAtLimitIsAllowed`	success at limit; failure +1 ¢	✅ Pass	2 ms
TC-12	`editedExpenseExcludesItsOwnPreviousAmountFromBudget`	success; ID 7 excluded	✅ Pass	1 ms
TC-13	`budgetOnlyLimitsExpenses`	success	✅ Pass	2 ms
TC-14	`expenseWithoutBudgetIsNotLimited`	success	✅ Pass	1 ms
TC-15	`repositoryErrorBecomesFailureInsteadOfCrash`	Result.failure("database closed")	✅ Pass	3 ms
TC-16	`amountAndDescriptionValidation`	true/false per partition	✅ Pass	0 ms
TC-17	`rangeValidationRejectsInvertedRanges`	false, false, true	✅ Pass	1 ms
TC-18	`nameAndPercentValidation`	true/false per boundary	✅ Pass	0 ms
TC-19	`parsesCommonInputsToCents`	exact cents	✅ Pass	0 ms
TC-20	`rejectsInvalidInputs`	null for each	✅ Pass	0 ms

Every case produced its expected result on the first run. No product defects were found in the validation unit, so no production code changed as a result of this run (see the change-summary document).

Figure 2. `ValidateTransactionUseCaseTest`: 15 of 15 tests passed. This covers TC-01 – TC-15; the 9 cases added for this plan appear alongside the 6 existing ones.

Class com.denariidolor.ValidateTransactionUseCaseTest

all > com.denariidolor > ValidateTransactionUseCaseTest

15	0	0	0.024s
tests	failures	ignored	duration

100%
successful

Tests

Test	Duration	Result
budgetOnlyLimitsExpenses	0.002s	passed
editedExpenseExcludesItsOwnPreviousAmountFromBudget	0.001s	passed
expenseExactlyAtLimitIsAllowed	0.002s	passed
expenseFailsWhenBudgetExceeded	0.001s	passed
expenseWithoutBudgetIsNotLimited	0.001s	passed
failsWhenCategoryOrAccountMissing	0.002s	passed
rejectsBlankDescription	0.001s	passed
rejectsNonPositiveAccountOrCategoryIds	0.001s	passed
rejectsNonPositiveDate	0.001s	passed
rejectsZeroOrNegativeAmount	0.003s	passed
repositoryErrorBecomesFailureInsteadOfCrash	0.003s	passed
transferFailsWhenDestinationAccountMissing	0.001s	passed
transferFailsWhenDestinationMatchesSource	0.002s	passed
transferRequiresDestination	0.002s	passed
validTransferPasses	0.001s	passed

Generated by Gradle 8.11.1 at Sep 21, 2026, 11:33:42 AM

Figure 3. `ValidatorsTest`: TC-16 – TC-18.

Class com.denariidolor.ValidatorsTest

all > com.denariidolor > ValidatorsTest

3	0	0	0.001s
tests	failures	ignored	duration

100%
successful

Tests

Test	Duration	Result
amountAndDescriptionValidation	0s	passed
nameAndPercentValidation	0s	passed
rangeValidationRejectsInvertedRanges	0.001s	passed

Generated by Gradle 8.11.1 at Sep 21, 2026, 11:33:42 AM

Figure 4. `MoneyTest`: TC-19 – TC-20, plus 2 formatting and precision tests.

Class com.denariidolor.util.MoneyTest

all > com.denariidolor.util > MoneyTest

4	0	0	0.001s
tests	failures	ignored	duration

100%
successful

Tests

Test	Duration	Result
centsSumExactly	0s	passed
formatsCents	0.001s	passed
parsesCommonInputsToCents	0s	passed
rejectsInvalidInputs	0s	passed

Generated by Gradle 8.11.1 at Sep 21, 2026, 11:33:42 AM

3. Coverage of the unit under test

Figure 5. JaCoCo coverage by class in `domain.usecase`. `ValidateTransactionUseCase` shows 96 % instruction and 94 % branch coverage.

debug > com.denariidolor.domain.usecase Source Files Sessions

com.denariidolor.domain.usecase

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
AccountUseCases		73%		63%	6	18	5	23	1	7	0	1
CategoryUseCases		86%		64%	9	25	0	26	0	8	0	1
UpdateTransactionUseCase		75%		71%	4	9	1	9	0	2	0	1
AddTransactionUseCase		69%		50%	4	6	1	7	0	2	0	1
SearchTransactionUseCase		0%		0%	3	3	3	3	2	2	1	1
ValidateTransactionUseCase		96%		94%	2	23	1	38	0	5	0	1
GenerateReportUseCase		97%		88%	3	14	0	31	1	5	0	1
DeleteTransactionUseCase		90%		83%	1	5	0	4	0	2	0	1
AccountUseCases.Companion		0%		n/a	1	1	1	1	1	1	1	1
CategoryUseCases.Companion		0%		n/a	1	1	1	1	1	1	1	1
BudgetUseCases		100%		100%	0	9	0	20	0	4	0	1
Total	260 of 1,909	86%	35 of 150	76%	34	114	11	161	6	39	3	11

Generated by the Android Gradle plugin 8.7.3 Created with JaCoCo 0.8.12.202403310830

Figure 6. Line-by-line coverage of `ValidateTransactionUseCase.kt`. Green means executed and a green diamond means all branches taken; yellow means some branches missed and red means not executed. Every `return failure(...)` line from the plan's decision flow is green.

```
30. import javax.inject.Inject
31.
32. class ValidateTransactionUseCase @Inject constructor(
33.     private val categoryRepository: CategoryRepository,
34.     private val accountRepository: AccountRepository,
35.     private val budgetRepository: BudgetRepository,
36.     private val transactionRepository: TransactionRepository
37. ) {
38.     suspend operator fun invoke(transaction: TransactionEntity): Result<Unit> {
39.         if (!Validators.isValidAmount(transaction.amountCents)) return failure("Invalid amount")
40.         if (!Validators.isValidDescription(transaction.description)) return failure("Description cannot be blank")
41.         if (!Validators.isValidDateEpoch(transaction.dateEpochMillis)) return failure("Invalid date")
42.         if (transaction.accountId <= 0L) return failure("Invalid account")
43.         if (transaction.categoryId <= 0L) return failure("Invalid category")
44.         if (transaction.type == TransactionType.TRANSFER) {
45.             val transferAccountId = transaction.transferAccountId ?: return failure("Transfer destination is required")
46.             if (transferAccountId == transaction.accountId) return failure("Transfer destination must be different")
47.         }
48.         return runSuspendCatching { checkReferencesAndBudget(transaction) }.getOrElse { Result.failure(it) }
49.     }
50.
51.     private suspend fun checkReferencesAndBudget(transaction: TransactionEntity): Result<Unit> {
52.         if (categoryRepository.getById(transaction.categoryId) == null) return failure("Category not found")
53.         if (accountRepository.getById(transaction.accountId) == null) return failure("Account not found")
54.         val transferAccountId = transaction.transferAccountId
55.         if (transaction.type == TransactionType.TRANSFER &&
56.             transferAccountId != null &&
57.             accountRepository.getById(transferAccountId) == null
58.         ) {
59.             return failure("Transfer destination account not found")
60.         }
61.         if (transaction.type == TransactionType.EXPENSE) {
62.             val budget = budgetRepository.getByCategoryId(transaction.categoryId)
63.             if (budget != null) {
64.                 val (start, end) = monthBounds(transaction.dateEpochMillis)
65.                 val spent = transactionRepository.getExpenseTotalForCategory(
66.                     categoryId = transaction.categoryId,
67.                     startInclusive = start,
68.                     endInclusive = end,
69.                     excludeTransactionId = transaction.id
70.                 )
71.                 if (spent + transaction.amountCents > budget.monthlyLimitCents) {
72.                     return Result.failure(IllegalStateException("Budget threshold violated"))
73.                 }
74.             }
75.         }
76.         return Result.success(Unit)
77.     }
78.
79.     private fun failure(message: String): Result<Unit> = Result.failure(IllegalArgumentException(message))
80.
81.     private fun monthBounds(epochMillis: Long): Pair<Long, Long> {
82.         val date = Instant.ofEpochMilli(epochMillis).atZone(ZoneId.systemDefault()).toLocalDate()
83.         return DateUtils.monthRangeEpochMillis(date.year, date.monthValue)
84.     }
85. }
```

Generated by the Android Gradle plugin 8.7.3 — Created with JaCoCo 0.8.12.202403310830

The three lines that are not fully green were analysed and accepted:

Line	JaCoCo	Explanation	Action
38 `suspend operator fun invoke(...)`	red (not covered)	A Kotlin compiler artifact. The coroutine state machine attributes synthetic continuation code to the signature line. The body (lines 39–48) is fully covered, so the function clearly ran.	None
48 `runSuspendCatching Ellipsis.getOrElse Ellipsis`	1 of 4 branches missed	The missed branch is the coroutine's *suspended-and-resumed* path. The in-memory fakes return immediately and never actually suspend. Both the success and failure outcomes are covered (TC-05, TC-15).	None; the path is exercised by the instrumented Room tests
56 `transferAccountId != null &&`	1 of 2 branches missed	A defensive check. A transfer with a `null` destination has already been rejected on line 45 (TC-06), so the `false` branch is unreachable.	Kept as a guard for future callers

Figure 7. Coverage of the supporting helpers in `util` (`Validators`, `Money`).

debug > com.denariidolor.util Source Files Sessions

com.denariidolor.util

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
DateUtils		70%		n/a	4	9	4	12	4	9	0	1
SessionManager		55%		66%	5	11	4	15	3	8	0	1
ResultExtKt		0%		n/a	1	1	7	7	1	1	1	1
Money		90%		87%	2	9	1	10	1	5	0	1
SeedAccount		75%		n/a	2	4	0	1	2	4	0	1
SessionManager_Factory.InstanceHolder		0%		n/a	1	1	1	1	1	1	1	1
SeedCategory		88%		n/a	1	4	0	1	1	4	0	1
Validators		100%		90%	3	21	0	14	0	6	0	1
Constants		100%		n/a	0	3	0	7	0	3	0	1
MoneyKt		100%		100%	0	5	0	7	0	2	0	1
Total	117 of 591	80%	6 of 49	87%	19	68	17	75	13	43	2	10

Generated by the Android Gradle plugin 8.7.3 Created with JaCoCo 0.8.12.202403310830

4. Full unit suite (regression)

Test class	Tests	Failures + errors	Skipped	Time
`DateUtilsTest`	3	0	0	0.020 s
`GenerateReportUseCaseTest`	3	0	0	0.305 s
`LedgerTest`	8	0	0	0.013 s
`ManageReferenceDataUseCaseTest`	8	0	0	0.046 s
`SearchFilterParserTest`	7	0	0	0.013 s
`SessionManagerTest`	2	0	0	0.002 s
`TransactionCrudUseCaseTest`	5	0	0	0.015 s
`TransactionEntityMappingsTest`	2	0	0	0.001 s
`TransactionModelTest`	4	0	0	0.003 s
`ValidateTransactionUseCaseTest`	15	0	0	0.026 s
`ValidatorsTest`	3	0	0	0.001 s
`data.local.db.DatabaseKeysTest`	5	0	0	0.033 s
`data.local.preferences.SecurityProfileServiceTest`	17	0	0	6.111 s
`domain.report.ReportFormattingTest`	4	0	0	0.023 s
`presentation.ui.CategoryIconsTest`	3	0	0	0.039 s
`presentation.ui.DashboardMappersTest`	12	0	0	0.014 s
`presentation.ui.FormMappingTest`	4	0	0	0.004 s
`presentation.ui.TransactionTypeDefaultsTest`	2	0	0	0.005 s
`presentation.viewmodel.DashboardViewModelTest`	1	0	0	0.165 s
`presentation.viewmodel.TransactionViewModelTest`	4	0	0	0.025 s
`util.MoneyTest`	4	0	0	0.001 s
Total	**116**	**0**	**0**	**6.865 s**

`SecurityProfileServiceTest` takes about 6 s because it runs real PBKDF2 hashing (210 000 iterations) on every PIN check. That cost is intentional.

Figure 8. The project-wide JaCoCo report for the JVM unit suite.

debug Sessions

debug

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.denariidolor.presentation.ui		1%		1%	560	571	555	575	150	157	60	62
com.denariidolor.presentation.ui.common		11%		2%	369	391	208	269	81	99	43	49
com.denariidolor.presentation.ui.dashboard		23%		11%	275	328	194	294	74	109	35	48
com.denariidolor.data.local.db.dao		0%		0%	184	184	647	647	143	143	49	49
com.denariidolor.presentation.ui.report		0%		0%	172	172	200	200	61	61	25	25
com.denariidolor.presentation.ui.search		8%		18%	155	175	159	180	68	71	24	25
com.denariidolor.presentation.ui.category		0%		0%	165	165	118	118	64	64	30	30
com.denariidolor.presentation.ui.auth		0%		0%	147	147	274	274	85	85	12	12
com.denariidolor		0%		0%	154	154	339	339	119	119	39	39
com.denariidolor.presentation.ui.account		0%		1%	90	92	87	88	40	41	17	18
com.denariidolor.data.local.db		0%		0%	53	53	211	211	28	28	7	7
com.denariidolor.presentation.ui.budget		4%		0%	80	83	81	84	31	34	13	15
com.denariidolor.data.export		0%		0%	33	33	107	107	23	23	6	6
com.denariidolor.data.repository		0%		0%	48	48	62	62	37	37	10	10
com.denariidolor.data.local.preferences		71%		62%	80	135	71	220	33	70	4	13
com.denariidolor.domain.usecase		86%		76%	34	114	11	161	6	39	3	11
com.denariidolor.domain.model		79%		80%	33	102	4	100	29	91	3	14
com.denariidolor.presentation.ui.transaction		76%		71%	29	61	7	85	17	38	6	15
com.denariidolor.data.local.db.security		60%		57%	14	28	16	31	7	15	2	3
com.denariidolor.util		80%		87%	19	68	17	75	13	43	2	10
com.denariidolor.di		0%		0%	9	9	15	15	8	8	4	4
hilt_aggregated_deps		0%		n/a	20	20	20	20	20	20	20	20
com.denariidolor.domain.report		91%		100%	3	16	2	34	3	11	0	2
com.denariidolor.presentation.ui.settings		0%		n/a	4	4	1	1	4	4	1	1
com.denariidolor.data.local.db.entity		96%		n/a	1	27	0	19	1	27	0	4
dagger.hilt.internal.processedrootsentinel.codegen		0%		n/a	1	1	1	1	1	1	1	1
dagger.hilt.internal.aggregatedroot.codegen		0%		n/a	1	1	1	1	1	1	1	1
Total	38,933 of 46,261	15%	3,023 of 3,448	12%	2,733	3,182	3,408	4,211	1,147	1,439	417	494

Generated by the Android Gradle plugin 8.7.3 Created with JaCoCo 0.8.12.202403310830

The project-wide figure (19 % of lines) counts only the **JVM unit tests**. Compose screens, Activities, Room DAOs and the SQLCipher database need an Android runtime, so the **instrumented** suite covers them instead (`connectedDebugAndroidTest`, 41 tests on API 26 and 35 in CI). The domain, validation and utility layers, which hold the business rules, have 77–96 % line coverage.

5. Conclusion

The validation unit meets every exit criterion in the test plan:

- All 20 planned cases passed.
- There are no regressions across the 116-test suite.
- The unit under test has 97.4 % line and 94.4 % branch coverage, and every failure path is exercised.

The remaining uncovered code is either compiler-generated or an unreachable defensive guard. The build is accepted for this test cycle.

Evidence retained:

- `app/build/test-results/testDebugUnitTest/*.xml` (JUnit XML)

- `app/build/reports/tests/testDebugUnitTest/` (HTML test report)
- `app/build/reports/coverage/test/debug/` (JaCoCo, including `report.xml`)